

Security Review

Cantina Managed

OPTIMISM: UPGRADE 20

Cantina Managed review by:

R0bert, Lead Security Researcher

Slowfi, Lead Security Researcher



August 20, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	5
3.1	Low Risk	5
3.1.1	Initial super deployments accept invalid anchors	5
3.1.2	<code>StandardValidator</code> accepts arbitrary same-version super-permissioned game implementations	5
3.1.3	U20 <code>StandardValidator</code> does not bind the <code>SystemConfig</code> chain ID to the validated super-game domain	6
3.1.4	Super-root migration can retain a legacy anchor and leave permissionless invalid claims unchallengeable	7
3.1.5	<code>SuperPermissioned</code> game initialization can trap a configured bond	8
3.1.6	The validator does not bind dispute-game registries to the Portal's registry	8
3.2	Informational	9
3.2.1	Initial permissionless deployments accept unavailable VM prestates	9
3.2.2	<code>Implementations</code> tuple removal can silently misdecode legacy container responses	10
3.2.3	U20 upgrades canonicalize divergent legacy fee-scalar state	10
3.2.4	<code>tx.origin</code> enforces an EOA-only proposer model and propagates authority through intermediaries	11
3.2.5	A standard member upgrade can restore legacy games to a shared interop set	11
3.2.6	A zero <code>SuperPermissionedDisputeGame</code> proposer cannot create games	12
3.2.7	U20 resets the retained <code>overhead</code> value during upgrade	12
3.2.8	Legacy validator input rejects healthy super-mode deployments	12



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

Optimism is a fast, stable, and scalable L2 blockchain built by Ethereum developers, for Ethereum developers. Built as a minimal extension to existing Ethereum software, Optimism's EVM-equivalent architecture scales your Ethereum apps without surprises. If it works on Ethereum, it works on Optimism at a fraction of the cost.

From Aug 6th to Aug 12th the Cantina team conducted a review of [optimism](#) on commit hash [7799a246](#). The team identified a total of **14** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	6	4	2
Gas Optimizations	0	0	0
Informational	8	1	7
Total	14	5	9

2.1 Scope

The security review had the following components in scope for [optimism](#) on commit hash [7799a246](#):

```
packages/contracts-bedrock
├── interfaces
│   ├── dispute
│   │   ├── ISuperFaultDisputeGame.sol
│   │   └── ISuperPermissionedDisputeGame.sol
│   └── L1
│       ├── IOptimismPortal2.sol
│       ├── ISystemConfig.sol
│       └── opcm
│           ├── IOPCContractsManagerContainer.sol
│           ├── IOPCContractsManagerMigrationValidator.sol
│           ├── IOPCContractsManagerMigrator.sol
│           └── IStandardValidatorUtils.sol
└── src
    ├── dispute
    │   ├── SuperFaultDisputeGame.sol
    │   ├── SuperPermissionedDisputeGame.sol
    │   └── lib
    │       ├── LibGameArgs.sol
    │       └── Types.sol
    ├── L1
    │   ├── OPContractsManagerStandardValidator.sol
    │   ├── OptimismPortal2.sol
    │   ├── SuperchainConfig.sol
    │   ├── SystemConfig.sol
    │   └── opcm
    │       ├── OPContractsManagerContainer.sol
    │       ├── OPContractsManagerUtils.sol
    │       ├── OPContractsManagerUtilsCaller.sol
    │       ├── OPContractsManagerV2.sol
    │       └── StandardValidatorUtils.sol
    └── libraries
        ├── Constants.sol
        └── DevFeatures.sol
```



```
|  └─ Features.sol  
└─ universal  
    └─ SafeSend.sol
```



3 Findings

3.1 Low Risk

3.1.1 Initial super deployments accept invalid anchors

Severity: Low Risk.

Context: `OPContractsManagerV2.sol`#L720-L790.

Description: `_assertValidFullConfig` accepts any nonzero initial anchor below `uint64.max`. When a permissionless game is enabled, it only excludes the exact `0xdead` placeholder. U20 commit `9714d4dd2c77052fad62505598db3568b1458f7d` (PR 21720) also newly permits `SUPER_CANNON_KONA` to be the respected game at initial deployment. The function never proves that the supplied root is the chain's SuperV1 genesis commitment or that its sequence is a timestamp the challenger can retrieve.

For example, a chain operator can accidentally supply a legacy VO output root and L2 block number. `OPContractsManagerV2.deploy`, the canonical `DeployOPChain` script, and `OPContractsManagerStandardValidator` all accept that tuple. A fresh `AnchorStateRegistry` then exposes it as the starting proposal for permissionless super games.

An arbitrary proposer can create a correctly encoded super game one sequence later. The production challenger rejects the starting block number as a pre-genesis timestamp, so it cannot create a player. The unopposed claim expires into `DEFENDER_WINS` and becomes claim-valid. Consequently, its attacker-selected per-chain output root can authorize fabricated withdrawals from all ETH or tokens later held by the portal and lockbox.

The official `op-deployer` prepare pipeline computes the correct genesis proposal, which reduces likelihood. The public OPCM API and supported Forge deployment script still accept the malformed tuple. The repository's own permissionless deployment fixture uses an arbitrary non-placeholder root and sequence.

Recommendation: Do not allow a fresh chain to respect a permissionless super game based only on a caller-supplied `Proposal1`. Bootstrap with `SUPER_PERMISSIONED`, then permit `SUPER_CANNON_KONA` only after a trusted, canonical super game has installed an authenticated genesis or later anchor. If direct permissionless bootstrap remains supported, require a proposal attested by the prepared genesis process and make both `DeployOPChain` and `OPContractsManagerStandardValidator` reject values that do not match it.

OP Labs: Acknowledged. The starting anchor is a trusted deployment input whose correctness is checked offchain by the operator and the supported `op-deployer` flow calculates it.

Cantina Managed: Acknowledged.

3.1.2 `StandardValidator` accepts arbitrary same-version super-permissioned game implementations

Severity: Low Risk.

Context: `StandardValidatorUtils.sol`#L492-L496.

Description: The U20 `StandardValidator` stores the release's approved `superPermissionedDisputeGameImpl` address and passes it into the new specialized SPDG validation path. However, `assertValidSuperPermissionedDisputeGame` does not compare the factory's registered implementation with that address. It calls `version()` on both contracts and accepts equality between the returned strings.

The registered candidate controls its own `version()` result. A different implementation can therefore report `1.0.0`, retain the canonical packed ASR and proposer arguments and make the validator return an empty success string. The other SPDG checks authenticate only the ASR and the proposer bytes stored by the factory; they do not prove that the registered implementation enforces those arguments or any game semantics.



This defeats the validator's stated purpose of ensuring that L1 contracts are configured correctly before and after an upgrade. The production SPDG enforces proposer authorization, well-formed calldata, root-claim binding to a decoded super-root preimage and a sequence newer than the anchor. A same-version substitute can omit all of these checks. Once cloned by the factory, it can allow an untrusted caller to create an immediately defender-winning game whose `rootClaimByChainId` returns an attacker-selected output root committing to fabricated MessagePasser state. After the normal ASR air gap, that game is claim-valid and can authorize fabricated withdrawal proofs through `OptimismPortal2`.

During an upgrade or manual recovery, the DGF owner can register an incorrect, compromised or lookalike implementation while retaining the canonical packed ASR and proposer arguments. The substitute reports the official `1.0.0` version, so `OPContractsManagerStandardValidator.validate(..., false)` returns an empty success string even though the implementation is different. An arbitrary account can then create a `DEFENDER_WINS` game with an attacker-selected output root. The factory registration, respected-game flag, resolution timestamp and normal ASR air gap make the game claim-valid, allowing the attacker to prove and finalize a withdrawal against the fabricated root and release portal or shared-lockbox assets. The condition requires a privileged implementation change, giving it low-to-moderate likelihood; exploitation after that change is permissionless and requires only the ordinary finality and proof delays.

Recommendation: Require `game.gameAddress == _impls.expectedGameImpl` in `assertValidSuperPermissionedDisputeGame`; do not use a value returned by the candidate itself as identity evidence. If operational requirements permit multiple approved builds, validate an immutable allowlist of exact runtime code hashes or implementation addresses. Apply the same identity rule to every standard and migration validator path and add a negative test where a different implementation returns the expected version and canonical getters.

OP Labs: Fixed in commit [283bb45](#).

Cantina Managed: Fix verified. `StandardValidator` now requires the factory's registered `SUPER_PERMISSIONED` implementation to equal the release-approved implementation address. A same-version lookalike is rejected with `SPDG-150`.

3.1.3 U20 `StandardValidator` does not bind the `SystemConfig` chain ID to the validated super-game domain

Severity: Low Risk.

Context: `OPContractsManagerStandardValidator.sol#L275-L304`, `OPContractsManagerStandardValidator.sol#L512-L514`, `OPContractsManagerStandardValidator.sol#L894`, `OPContractsManagerStandardValidator.sol#L911-L933`.

Description: The U20 `StandardValidator` accepts the expected L2 chain ID as caller-supplied `ValidationInputDev.l2ChainId`, but the new super-permissioned-game branch returns through `assertValidSuperPermissionedDisputeGame` without comparing that value with `SystemConfig.l2ChainId()`. The general `SystemConfig` validation at lines 285-303 also does not perform this comparison. Super-game implementations deliberately store zero rather than a member chain ID, so the remaining game-argument checks cannot establish the missing binding.

This matters because `OptimismPortal2.proveWithdrawalTransaction` selects a member output root by calling `rootClaimByChainId(systemConfig.l2ChainId())`. A validator invocation for chain X can therefore return an empty success string while the Portal will use chain Y's root. If Y is absent from the super root, withdrawals for the affected Portal revert. If Y is present, the Portal checks withdrawals against Y's MessagePasser root rather than X's, creating a wrong-domain settlement configuration and potentially allowing the same Y withdrawal evidence to be settled through more than one Portal.

For example, an imported deployment or reinitialization can store chain ID Y in `SystemConfig` while the validator input and intended domain use chain ID X. The DGF, ASR, implementations, proposer and all other checked fields can remain canonical. U20 validates the super-game configuration without comparing X and Y, so it returns an empty success string. The Portal later selects Y from every super root. It either cannot find Y and blocks withdrawals or verifies an X Portal withdrawal against Y's



MessagePasser state. Under a malformed shared topology, the same Y withdrawal evidence could therefore be settled through more than one Portal using pooled liquidity. This requires a malformed deployment, reinitialization or operator-supplied validation input rather than an unprivileged setter. Canonical tooling normally derives the values from one intent, so the mismatch has low likelihood. The validator is expressly intended to catch this configuration error, but validation is advisory and trusted governance can repair it.

Recommendation: In `assertValidSystemConfig`, require `_sysCfg.l2ChainId() == _input.l2ChainID` by passing the expected chain ID into the helper. Keep the existing legacy-game argument check as an additional predicate.

OP Labs: Fixed in commit [cba37e8](#).

Cantina Managed: Fix verified. `StandardValidator` now requires `SystemConfig.l2ChainId()` to equal the chain ID supplied for validation and reports `SYSCON-140` on a mismatch.

3.1.4 Super-root migration can retain a legacy anchor and leave permissionless invalid claims unchallengeable

Severity: Low Risk.

Context: `OPContractsManagerV2.sol#L324-L344`, `OPContractsManagerV2.sol#L362-L367`, `OPContractsManagerV2.sol#L671-L678`, `OPContractsManagerV2.sol#L720-L729`, `DevFeatures.sol#L45-L61`.

Description: U20 changes `SUPER_ROOT_GAMES_MIGRATION` from opt-in to unconditionally enabled and extends the release allowance for `overrides.cfg.startingAnchorRoot`, but never requires the instruction when an upgrade disables the legacy output-root games and enables a super-root game. `_loadFullConfig` falls back to `AnchorStateRegistry.getStartingAnchorRoot()` when that instruction is absent. This is not the current anchor when `anchorGame` is set: the current anchor is the legacy game's output root and L2 block number. `_assertValidFullConfig` validates the anchor only for initial deployments, so the upgrade and the post-upgrade `StandardValidator` both accept this mixed state.

The subsequent `AnchorStateRegistry.initialize` call receives the unchanged original starting root. Its `rootChanged` check is therefore false and it does not clear the existing legacy `anchorGame`. New `SuperFaultDisputeGame` instances consequently copy the legacy output-root hash and L2 block number into `startingProposal`, although the super trace interprets that number as a timestamp and the hash as a super-root commitment.

This breaks the honest challenger before any onchain move is made. The challenger constructs its top prestate with `SuperRootAtTimestamp(legacyL2BlockNumber)`. For a normal OP chain, the legacy block number is far below the L2 genesis Unix timestamp, so `op-node` rejects it in `TargetBlockNumber` as pre-genesis. That provider error is not `ErrInvalidPrestate`; the scheduler therefore refuses to create the game player even when `allow-invalid-prestate` is enabled.

Because of this issue, after an accepted but incomplete U20 migration, any account can create a permissionless `SUPER_CANNON_KONA` game with an arbitrary, correctly encoded super root at sequence number `N + 1`. Because honest challengers cannot instantiate a trace for the retained legacy prestate, no counter is posted. Once the game clock expires it resolves `DEFENDER_WINS`; after the finality delay, the `AnchorStateRegistry` treats it as a valid anchor. The attacker's per-chain output root can then be used to prove fabricated withdrawals through `OptimismPortal2`, putting all ETH and tokens secured by that portal at risk.

The precondition for this issue to be triggered is an otherwise authorized U20 migration that installs the super game family but omits one optional-looking extra instruction, followed by normal guardian activation of the permissionless super game (or a migration that selects it immediately). For example, suppose a permissionless chain has a finalized legacy output-root game as its anchor at L2 block `N`. If the chain's `ProxyAdmin` owner disables legacy games, enables the super game family and omits `overrides.cfg.startingAnchorRoot`, `OPContractsManagerV2.upgrade` succeeds, retains the legacy `anchorGame` and the `StandardValidator` returns no errors. Once the guardian



selects `SUPER_CANNON_KONA` as respected, an unprivileged attacker can create a super game containing an attacker-selected output root at timestamp `N + 1` and pay the configured initial bond. Honest challengers query `superroot_atTimestamp(N)`, but `op-node` rejects `N` as pre-genesis, so the scheduler never creates a player. The attacker waits for the game clock and finality delay, resolves the uncontested root as `DEFENDER_WINS`, closes the game and uses the now-valid anchor to prove and finalize withdrawals against the attacker-selected per-chain output root. This is a realistic configuration failure rather than a compromised-admin attack: the code comment says the migration requires the override, the public upgrade API does not enforce that requirement and the `StandardValidator` reports the resulting system as valid. The repository's own `SuperGameTestInit` performs this exact omission. The official fork migration helper includes the instruction, which reduces likelihood and the guardian has the dispute/finality window to detect and blacklist a malicious game.

Recommendation: When an upgrade transitions from a legacy respected game type to any super-root game type, require a `startingAnchorRoot` instruction instead of merely permitting it. Validate that the supplied root is nonzero, that its sequence number is a valid timestamp strictly greater than the current `getAnchorRoot()` sequence and ensure the legacy `anchorGame` is cleared atomically. The `StandardValidator` should also reject a super-root configuration whose active anchor is still a legacy game type. If feasible, have migration tooling supply the encoded super-root preimage and verify its hash, rather than accepting an unbound hash alone.

OP Labs: Acknowledged. OPCM cannot fully validate a new anchor onchain. OP Labs will instead verify that an anchor root is set and matches a canonical, finalized block through the `superchain-ops` upgrade task or a manual verification process.

Cantina Managed: Acknowledged.

3.1.5 `SuperPermissioned` game initialization can trap a configured bond

Severity: Low Risk.

Context: `SuperPermissionedDisputeGame.sol#L45-L62`, `OPContractsManagerV2.sol#L749-L764`.

Description: The standalone `SuperPermissionedDisputeGame.initialize` function is payable, but the game has no bond accounting, distribution, or withdrawal path for an initialization value. A nonzero initialization bond sent to a `SuperPermissioned` clone therefore remains trapped in that clone.

The normal U20 configuration validator checks that `SUPER_PERMISSIONED` has a zero initialization bond, but the game implementation itself does not enforce that invariant. Any configuration path that supplies a nonzero bond can consequently create publicly usable game clones that hold unrecoverable ETH. Malformed privileged game configuration can permanently lock ETH in each created `SuperPermissioned` clone. The amount is bounded per clone, but the condition is repeated for every game created under the affected configuration.

Recommendation: Make the `SuperPermissioned` initializer nonpayable if it must never receive a bond, or explicitly reject nonzero `msg.value` and enforce the zero bond invariant at every configuration mutation path. Add a balance conservation test for clone creation.

OP Labs: Fixed in commit `44eb954`.

Cantina Managed: Fix verified. `SuperPermissionedDisputeGame.initialize` now rejects any nonzero `msg.value` with `IncorrectBondAmount` before completing initialization.

3.1.6 The validator does not bind dispute-game registries to the Portal's registry

Severity: Low Risk.

Context: `OPContractsManagerStandardValidator.sol#L902-L933`.

Description: The standard validator selects super mode from the Portal's `AnchorStateRegistry`, but decodes an `AnchorStateRegistry` independently from each dispute game factory's encoded arguments. It validates each decoded registry's implementation, `ProxyAdmin`, `SystemConfig`, DGF,



retirement timestamp, and anchor, but never requires the decoded registry to equal the Portal's registry.

An authorized configuration can therefore use registry A for the Portal while the `SuperPermissioned` game arguments point to a second ordinary registry B. If both registries pass their local checks but respect different game types, standard validation can return no errors for the split. A proposer can then create a game that snapshots B's respected type while the Portal consults A, leaving the game and Portal registry identity unbound. A trusted configuration mistake can therefore make `StandardValidator` report success for a registry split that undermines the intended dispute game and withdrawal trust boundary.

Recommendation: Read `portal.anchorStateRegistry()` once and require every enabled game's encoded registry to equal it. Validate the Portal registry through the same deep checks, and add a negative test where two locally valid registries have different respected game types.

OP Labs: Fixed in commit [ef695a3](#).

Cantina Managed: Fix verified. `AnchorStateRegistry` validation now requires the registry encoded by each game to equal `OptimismPortal2.anchorStateRegistry()` and reports `ANCHORP-70` on a mismatch. The release covers legacy, super, and ZK game paths with negative regression tests.

3.2 Informational

3.2.1 Initial permissionless deployments accept unavailable VM prestates

Severity: Informational.

Context: [OPContractsManagerV2.sol#L767-L798](#).

Description: U20 allows a fresh chain to enable and immediately respect `SUPER_CANNON_KONA`, but `_assertValidFullConfig` accepts its fault-VM absolute prestate whenever the commitment is nonzero. It does not require an approved release commitment or prove that a matching Cannon/Kona state artifact is available. The supported `DeployOPChain` wrapper applies the same nonzero check. `OPContractsManagerStandardValidator` then compares the installed value with another caller-supplied copy, so it also reports the configuration as valid.

The production challenger must resolve the exact onchain commitment before it constructs a type-9 game player. It checks local files and the configured prestate URL for `.bin.gz`, `.json.gz` and `.json` artifacts, then recomputes the commitment. An unavailable or mismatched artifact returns `ErrPrestateUnavailable` before player construction. The scheduler's optional invalid-prestate exception is ineffective because it is applied later and only accepts `ErrInvalidPrestate`.

For example, an operator can make a one-word typo in the selected prestate or supply a stale unpublished commitment. All deployment and validation calls succeed. Once the chain is funded, an untrusted proposer can create a game containing a fabricated per-chain output root. No standard challenger can construct a player. The root claim therefore expires to `DEFENDER_WINS`, becomes an ASR anchor after finality and authorizes a withdrawal that never occurred on L2.

The impact is an arbitrary loss of ETH or tokens held by the portal or shared lockbox. The precondition is an honest deployment configuration error, not malicious governance. U20's `op-deployer` prestate command explicitly accepts arbitrary 32-byte CLI, environment and per-chain override values other than one reserved placeholder, so this error is realistic. Exploitation after deployment is permissionless.

Recommendation: Do not let a fresh chain respect a permissionless game unless the selected absolute prestate is authenticated and available. Bind each permitted game type to an immutable allowlist of release-approved prestate commitments in OPCM and `StandardValidator`. Before broadcasting deployment, make `op-deployer` download or load the selected artifact, recompute its commitment with the production state converter and reject any mismatch or unavailable file.

If arbitrary prestate overrides must remain supported, initially respect `SUPER_PERMISSIONED`. Enable `SUPER_CANNON_KONA` only after an end-to-end challenger dry run has resolved the artifact and constructed the game player.



OP Labs: Acknowledged. The contracts intentionally validate the prestate commitment, while artifact availability and correctness remain part of the trusted release-approval process.

Cantina Managed: Acknowledged.

3.2.2 Implementations tuple removal can silently misdecode legacy container responses

Severity: Informational.

Context: `IOPContractsManagerContainer.sol#L13-L16`.

Description: U20 removes `protocolVersionsImpl` from the second position of the all-static `Implementations` return tuple. Return types do not contribute to a function selector, so the `implementations()` selector remains unchanged across the release boundary.

A caller compiled against the U20 interface can therefore call a pre-U20 container and successfully decode its 19-word response as the new 18-field struct. The decoder consumes the first 18 words and ignores the final word: `11ERC721BridgeImpl` receives the old `protocolVersionsImpl`, every following field is shifted, `storageSetterImpl` receives the old `zkDisputeGameImpl` and the actual old `storageSetterImpl` is discarded.

No target production path was found that queries a legacy container through the new interface, so this is not a demonstrated protocol exploit. It is an informational compatibility hazard for release-spanning deployment, inventory, validation or monitoring tooling that selects an ABI independently of the container version.

Recommendation: Use a versioned getter with a distinct selector for the new tuple or query and validate the container version before selecting the return schema. Low-level compatibility tooling should require the exact expected return length before decoding. Regenerate dependent ABIs during the U20 migration and add negative tests for both old-caller/new-container and new-caller/old-container combinations.

OP Labs: Acknowledged. Supported deployment tooling upgrades the ABI and contracts together and the opposite stale-ABI direction reverts safely.

Cantina Managed: Acknowledged.

3.2.3 U20 upgrades canonicalize divergent legacy fee-scalar state

Severity: Informational.

Context: `OPContractsManagerV2.sol#L626-L643`.

Description: U20's upgrade configuration preserves `basefeeScalar()` and `blobbasefeeScalar()` separately. `SystemConfig.initialize` then passes those values to `_setGasConfigEcotone`, which reconstructs the packed `scalar()` value rather than preserving its prior raw value.

Before U20, the deprecated `setGasConfig(uint256,uint256)` function could update `scalar()` without updating the two mirror fields. A chain that used that setter can therefore enter the upgrade with divergent values, after which U20 replaces `scalar()` with the version-1 encoding derived from the mirrors. A target-revision upgrade proof of concept reproduces this state transition.

The U20 release notice explicitly describes this possible divergence and the code intentionally treats the mirror fields as canonical, so the behavior is not a security vulnerability. It remains operationally relevant because the upgrade changes an observable configuration value on affected legacy deployments and may surprise tooling that expected raw storage preservation.

Recommendation: Have upgrade tooling compare the current `scalar()` value with the value reconstructed from `basefeeScalar()` and `blobbasefeeScalar()` before execution. Surface any mismatch and the exact post-upgrade value for explicit operator acknowledgement.

OP Labs: Acknowledged. No chain included in this upgrade used the deprecated setter and the component scalar fields are intentionally treated as canonical.



Cantina Managed: Acknowledged.

3.2.4 `tx.origin` enforces an EOA-only proposer model and propagates authority through intermediaries

Severity: Informational.

Context: `SuperPermissionedDisputeGame.sol#L45-L60`.

Description: The rewritten standalone `SuperPermissionedDisputeGame.initialize` authorizes creation with `tx.origin == proposer`. If the configured proposer is a Safe, module, account abstraction wallet, relayer, or another contract, the check can never pass because `tx.origin` is always an EOA. This creates a proposal liveness failure for supported contract based proposer flows.

When the proposer is an EOA, the authority is implicitly propagated through every intermediary contract it calls. An intermediary can create a defender winning game while the EOA remains the transaction origin. The game resolves immediately to `DEFENDER_WINS`, subject to the normal ASR respect, blacklist, and finality controls. The configured proposer account model may therefore be unusable, or an intermediary in the proposer call graph may gain unintended authority to create a resolved proposal.

Recommendation: Replace `tx.origin` authorization with an explicit supported account model. Use `msg.sender` for direct proposer calls, or verify an EIP 1271, signature, or module authorization binding the proposer, payload, chain, nonce, and expiry. Test EOA, Safe, module, account abstraction, and relayer call graphs.

OP Labs: Acknowledged. `msg.sender` is always the `DisputeGameFactory`, so the contract intentionally uses `tx.origin` for proposer authorization. Checking `gameCreator()` instead would add more code to this critical permission check, while `op-proposer` supports only EOAs and the existing `PermissionedDisputeGame` uses the same model.

Cantina Managed: Acknowledged.

3.2.5 A standard member upgrade can restore legacy games to a shared interop set

Severity: Informational.

Context: `OPContractsManagerV2.sol#L767-L815`.

Description: The ordinary `OPContractsManagerV2.upgrade` path accepts caller supplied dispute game configurations. Its legacy versus super mode restriction applies only to initial deployments, and the respected game override accepts any enabled listed type. On a shared interop set, the upgrade reinitializes the shared `AnchorStateRegistry` and rewrites the shared dispute game factory. U20 per proxy admin routing makes those shared writes succeed even when a non first member initiates the upgrade.

An authorized but malformed upgrade can therefore install and respect a legacy per chain game across every portal sharing the ASR and DGF. For legacy games, `OptimismPortal2` uses the raw root claim and does not exercise the legacy game's chain ID check. Withdrawal and output root hashes omit the chain ID, while replay state is local to each portal. The same chain A withdrawal proof can consequently be accepted through multiple portals and paid from shared custody. A privileged malformed upgrade can create cross portal withdrawal replay and duplicate value paid from a shared ETH lockbox. After the malformed state is installed, an ordinary proof submitter can trigger the duplicate finalization.

Recommendation: Make mode compatibility an invariant of every state mutating deployment and upgrade path. When the ASR and DGF are shared, reject all legacy respected types and legacy implementations, and preserve shared state only from a canonical full set configuration. Add a two member negative test that attempts legacy types from both member upgrade entry points and verifies that a withdrawal can be accepted by at most the source chain portal.

OP Labs: Acknowledged. The `migrate` function will not be called as part of U20, so chains will not share the `DisputeGameFactory` and `AnchorStateRegistry` components required for this scenario.



It would also take a privileged operation to install the invalid configuration and that same authority could more directly set an anchor against which invalid withdrawals could be proven.

Cantina Managed: Acknowledged.

3.2.6 A zero `SuperPermissionedDisputeGame` proposer cannot create games

Severity: Informational.

Context: `OPContractsManagerV2.sol#L733-L800`.

Description: `_assertValidFullConfig` allows an enabled `SUPER_PERMISSIONED` game with `proposer == address(0)`. The factory stores this value and the `StandardValidator` also accepts it when the expected proposer is zero.

`SuperPermissionedDisputeGame.initialize` requires `tx.origin == proposer()`. Since no usable transaction can originate from the zero address, every game creation reverts with `BadAuth`. If this is the respected game type, no permissioned super-root games can be created until governance repairs the configuration.

This requires a trusted `ProxyAdmin` owner to supply the bad value. The standard deployment script also rejects a zero proposer, so the issue is limited to configuration hardening.

Recommendation: Reject a zero proposer in `_assertValidFullConfig` and the `StandardValidator`.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.

3.2.7 U20 resets the retained `overhead` value during upgrade

Severity: Informational.

Context: `SystemConfig.sol#L240-L242`.

Description: U20 keeps the deprecated `overhead()` getter for integration compatibility and the release notice states that its stored value does not change during the upgrade. However, the reinitializer unconditionally sets `overhead` to zero. Any legacy nonzero value is therefore lost during an ordinary U20 upgrade.

This may surprise integrations that still read the getter. It does not affect fees or finality because `op-node` no longer uses this value after Ecotone.

Recommendation: Remove the `overhead = 0` write so legacy values remain unchanged. If the reset is intentional, update the release notice and integration guidance instead.

OP Labs: Fixed in commit [630e797](#).

Cantina Managed: Fix verified. The Upgrade 20 notice now states that the upgrade clears `overhead` to zero, that it remains zero, and that integrations reading the getter must expect this change. This documentation-only fix merged after the `op-contracts/v8.0.0-rc.3` tag, as intended and does not require a new contract release.

3.2.8 Legacy validator input rejects healthy super-mode deployments

Severity: Informational.

Context: `DevFeatures.sol#L56-L61`.



Description: The legacy `ValidationInput` provides only one prestate. `_toValidationInputDev` assigns it to Cannon and sets the expected Kona prestate to zero. U20 enables super-root mode by default, so validation compares this zero value against the nonzero prestate of `SUPER_CANNON_KONA`.

Consequently, the legacy overload reports a healthy deployment as invalid with `SCKDG-40` or reverts when failures are not allowed. The complete `ValidationInputDev` overload validates the same deployment correctly. This is a read-only compatibility issue with no direct custody impact.

Recommendation: Deprecate the legacy overload or return a clear unsupported-input error in super mode. Alternatively, obtain the expected Kona prestate from an authenticated configuration.

OP Labs: Acknowledged.

Cantina Managed: Acknowledged.